



Cairo University

Faculty of Computers and Artificial Intelligence

AI322: Supervised Learning

Project: Phase 1&2

IDs: 20230541_20230085_20231042_20230317_20230272

Course Instructor:

Dr. Ghada Dahy

1. Summary:

This report provides a comprehensive, expanded analysis of the German Traffic Sign Recognition Benchmark (GTSRB) project. The project is divided into two phases. Phase 1 focuses on classical computer vision pipelines, involving rigorous Exploratory Data Analysis (EDA), handcrafted feature engineering (HOG and Color Histograms), baseline model wrappers, and hyperparameter optimization for ensemble methods. Phase 2 transitions into deep representation learning, detailing the implementation of a custom Convolutional Neural Network (CNN), sophisticated feature map visualization for interpretability, transfer learning benchmarks, and autoencoder-based dimensionality reduction.

The overarching objective is to establish a robust, end-to-end pipeline capable of high-accuracy classification while maintaining computational efficiency and model transparency.

2. Introduction & Dataset Profile

The ability for autonomous vehicles and Advanced Driver Assistance Systems (ADAS) to accurately identify and interpret traffic signs is a critical requirement for safe navigation. The GTSRB dataset presents a highly realistic challenge, consisting of over 50,000 images across 43 distinct classes.

2.1 Challenges in the Dataset

Traffic sign recognition in the wild is complicated by several environmental factors:

- **Varying Illumination:** Images range from overexposed daylight scenarios to underexposed nighttime shots.
- **Motion Blur and Artifacts:** Many samples suffer from camera motion blur or compression artifacts.
- **Occlusions and Vandalism:** Signs may be partially blocked by foliage, other vehicles, or stickers.
- **Class Imbalance:** The real-world frequency of signs is heavily skewed. Speed limit signs (e.g., 50 km/h) are ubiquitous, while specific warning signs (e.g., dangerous curve) are rare.

3. Phase 1: Classical Machine Learning & Pipeline Engineering

Phase 1 establishes the baseline performance using algorithms that rely on manual feature extraction. This phase serves to highlight the limits of traditional computer vision techniques when applied to highly variable real-world data.

3.1 Exploratory Data Analysis (`eda_initial_notebook.ipynb`)

The foundation of the pipeline was established in the EDA notebook. Before feeding data into mathematical models, it is crucial to understand its statistical distribution.

Class Distribution Analysis:

The EDA process began with visualizing the class distributions using histogram plots. The severe imbalance in the dataset was immediately apparent. If standard accuracy were used as the sole metric, a model could artificially inflate its performance by simply predicting the majority class. Consequently, the evaluation framework was designed to prioritize the Macro F1-Score, which calculates the metric independently for each class and then takes the unweighted mean, ensuring minority classes contribute equally to the final evaluation score.

Sample Visualization and Preprocessing:

Randomly sampling and visualizing images from each class allowed for a qualitative assessment of intra-class variance. To handle variations in lighting, preliminary dimensionality reduction experiments were conducted using Principal Component Analysis (PCA). PCA helped visualize the dataset's variance in 2D and 3D space, revealing that raw pixel data alone contains too much overlapping noise to be linearly separable, thus motivating the need for robust feature engineering.

3.2 Feature Extraction System (`feature_extraction.py`)

Raw pixel values are highly sensitive to shifts in lighting, translation, and rotation. To create a robust classical machine learning model, a FeatureExtractor module was implemented to transition the data from the pixel domain into a highly discriminative feature space.

To ensure the features are invariant to the overall pixel count (making it robust to resizing), the histogram is normalized:

- $\text{feat_normalized} = \text{feat} / \Sigma(\text{feat})$

3.3 Baseline Model Architecture (baselines.py & pipeline_example.py)

To establish a benchmark, a reusable wrapper class `BaselineModels` was implemented. This class encapsulates data preparation, scaling, dimensionality reduction, training, and evaluation APIs. Two foundational algorithms were utilized:

K-Nearest Neighbors (KNN):

KNN is a non-parametric, instance-based learning algorithm. It classifies a new sample based on the majority class of its 'K' closest neighbors in the feature space. Because KNN relies on distance metrics (typically Euclidean distance: $d = \sqrt{(\Sigma(x_i - y_i)^2)}$), it is hyper-sensitive to the scale of the input features. The `BaselineModels` wrapper includes a `StandardScaler` integration.

- Standardization formula: $z = (x - \mu) / \sigma$
This ensures that the HOG features and Color Histogram features, which exist on different numerical scales, contribute equally to the distance calculation.

Gaussian Naive Bayes (GNB):

GNB provides a probabilistic baseline. It relies on Bayes' Theorem, with the "naive" assumption that all features are conditionally independent given the class.

- Bayes Theorem: $P(y | X) = [P(X | y) * P(y)] / P(X)$
Because the extracted features (especially PCA-reduced features) are continuous, GNB assumes they follow a normal (Gaussian) distribution.

The `pipeline_example.py` script provides an end-to-end demonstration, taking raw or synthetic image arrays, automatically routing them through the feature extractor, splitting the data via `train_test_split`, and evaluating the baseline models using Accuracy, Precision, Recall, Macro F1, and Confusion Matrices.

3.4 Hyperparameter Optimization (grid_search.py)

To maximize the potential of classical algorithms, systematic hyperparameter optimization was performed. The `grid_search.py` module leverages `sklearn.model_selection.GridSearchCV` to perform exhaustive searches over specified parameter spaces.

To prevent data leakage and manage computational complexity, Pipeline objects were constructed. A typical pipeline included:

1. `StandardScaler()`
2. `PCA()` (Principal Component Analysis)
3. The Classifier (e.g., `GradientBoostingClassifier`)

Grid Search Configurations:

The search space optimized several parameters across different models:

- **Gradient Boosting:** Optimized over the number of boosting stages (`n_estimators`: 50, 100) and the contribution of each tree (`learning_rate`: 0.1, 0.2).
- **Cross-Validation:** To validate performance safely, 3-fold cross-validation was employed. The dataset is divided into 3 equal parts; the model trains on 2 parts and validates on the 3rd, rotating until all parts have been tested.
- **Scoring Metric:** `f1_macro` was explicitly set as the optimization target to ensure the search prioritized models that performed well across all 43 classes, not just the majority classes.

3.5 Ensemble Methods (ensemble.py)

Individual classical models often struggle to generalize across highly complex datasets due to high bias (underfitting) or high variance (overfitting). Phase 1 concludes with the implementation of an `Ensemble_models` class to combine the predictive power of multiple algorithms.

1. Bagging (Random Forest):

Random Forest operates by building multiple decision trees during training. Each tree is trained on a random subset of the data

(bootstrapping) and a random subset of features. By averaging the predictions of these diverse, uncorrelated trees, the Random Forest drastically reduces the overall variance of the model, preventing overfitting on specific feature anomalies.

2. Boosting (Gradient Boosting & AdaBoost):

Unlike Bagging, Boosting builds trees sequentially.

- **AdaBoost (Adaptive Boosting):** Assigns higher weights to misclassified instances after each tree is built. Subsequent trees are forced to focus on the hardest-to-classify images.
- **Gradient Boosting:** Instead of adjusting weights, each new tree is trained to predict the residual errors of the combined previous trees, optimizing a differentiable loss function via gradient descent.

3. Soft Voting Classifier:

The climax of the ensemble approach is the VotingClassifier. Configured for 'soft' voting, it aggregates the probabilistic outputs of the underlying models (Random Forest, Gradient Boosting, AdaBoost).

Final Prediction Formula: $\hat{y} = \text{argmax}(\sum w_j * p_{ij})$

Where p_{ij} is the predicted probability of class i by classifier j . This synergistic approach ensures that the weaknesses of one classifier are compensated for by the strengths of another, resulting in the highest accuracy achievable within Phase 1.

4. Phase 2: Deep Learning, Architectures & Interpretability

4.1 Convolutional Neural Network Design (CNN_model.py)

At the core of Phase 2 is the TrafficSignCNN, a PyTorch-based architecture specifically tailored for the 32x32 spatial dimensions of the GTSRB dataset. The architecture is composed of distinct sequential blocks designed to downsample spatial resolution while expanding feature depth.

Convolutional Block 1:

- **Input:** 3 channels (RGB image), 32x32 pixels.

- **Layer:** Conv2d with 32 output filters, a 3x3 kernel, and padding of 1. The padding ensures the spatial dimensions remain 32x32 after the convolution.
- **Activation:** Rectified Linear Unit (ReLU), defined as $f(x) = \max(0, x)$, which introduces the non-linearity necessary for learning complex patterns.
- **Pooling:** MaxPool2d with a 2x2 kernel and stride of 2. This reduces the spatial dimensions by half, passing a 32-channel 16x16 tensor to the next block, providing translational invariance.

Convolutional Block 2:

- Takes the 32x16x16 tensor and applies a second Conv2d layer expanding to 64 filters.
- Followed by ReLU and 2x2 Max Pooling, yielding a 64x8x8 tensor.

Convolutional Block 3:

- Takes the 64x8x8 tensor and applies a third Conv2d layer expanding to 128 filters.
- Followed by ReLU and 2x2 Max Pooling, yielding a final feature map of 128x4x4.

Fully Connected (Dense) Block:

- **Flattening:** The 3D tensor is unrolled into a 1D vector of size 2048 ($128 * 4 * 4$).
- **Dense Layer 1:** A linear transformation fc1 mapping 2048 inputs to 512 hidden neurons.
- **Regularization:** A Dropout layer with probability $p=0.5$ is applied. During training, 50% of the neurons are randomly deactivated, forcing the network to learn redundant, robust features and drastically reducing overfitting.
- **Output Layer:** The final fc2 layer maps the 512 features to the 43 discrete class scores.

4.2 Training Dynamics (phase2_cnn_train.ipynb)

The training loop for the CNN was engineered to maximize generalization. Raw image data was passed through `torchvision.transforms` for data augmentation.

The network was trained using:

- **Loss Function:** Multi-class Cross-Entropy Loss.
 - Formula: $L = - \sum y_c * \log(p_c)$
 - This function intrinsically combines a Softmax activation with Negative Log-Likelihood, penalizing the model heavily when it is confident but incorrect.
- **Evaluation:** The training pipeline tracks both loss and accuracy across epochs. A comprehensive confusion matrix was plotted post-training to visualize the network's performance, specifically looking for common failure modes (e.g., confusing "Speed limit 30" with "Speed limit 50" due to visual similarities).

4.3 Model Interpretability and Feature Visualization (`cnn_feature_visualization.py`)

A persistent criticism of Deep Learning is the "black box" nature of neural networks. To address this, `CNN_model.py` was extended with an `extract_feature_maps` method, allowing the isolation of intermediate tensor states. The `CNNFeatureMapVisualizer` class provides four critical modes of interpretability:

1. **Learned Filter Visualizations:** By plotting the direct weight matrices of the first convolutional layer (`conv1.weight`), we can visibly inspect what the network is searching for. These typically manifest as Gabor-like filters—edge detectors responding to specific angles and color blobs.
2. **Layer-wise Feature Maps:** Passing a traffic sign through the network and visualizing the outputs of `conv1`, `conv2`, and `conv3`. This illustrates the hierarchical learning process: `conv1` highlights raw edges, `conv2` detects shapes (circles, triangles), and `conv3` activates on highly abstract, class-specific concepts.
3. **Aggregated Activation Heatmaps:** Summing the activations across all channels within a layer provides a thermal map of the network's attention, proving that the model is looking at the actual sign and not memorizing background artifacts (like trees or sky).

4. **Prediction Confidence Plots:** The visualizer plots the top-k Softmax probabilities, allowing researchers to analyze not just the predicted class, but the model's runner-up hypotheses and overall certainty.

4.4 Transfer Learning: VGG16

To establish a modern industry baseline, Transfer Learning was explored in `phase2_cnn_train.ipynb`. Transfer Learning leverages models pre-trained on massive datasets (like ImageNet, containing millions of images) to solve specialized tasks with minimal data.

VGG16: A classic, heavy architecture consisting of deep stacks of 3x3 convolutions. It provides excellent feature extraction but requires significant memory and computational overhead.

The transfer learning protocol utilized a two-phase approach:

1. **Warm-up Phase:** The pre-trained convolutional base was "frozen" (gradients disabled). A new, randomly initialized fully connected classification head was attached and trained. This prevents the large error gradients of the untrained head from destroying the fragile, pre-learned weights in the convolutional layers.
2. **Fine-Tuning Phase:** The convolutional base was "unfrozen," and the entire network was trained simultaneously with an exceptionally low learning rate (e.g., $1e-5$). This gently aligns the ImageNet-learned features to the specific visual domain of German traffic signs.

4.5 Optimizer Benchmarking and Learning Rate Schedulers (`trainer_opt.py`)

The mechanism by which a neural network updates its weights—the optimizer—plays a crucial role in convergence speed and final accuracy. A configurable trainer utility was implemented to ablate various optimizers:

- **SGD (Stochastic Gradient Descent):** Updates weights based purely on the current batch's gradient. Reliable but slow to navigate flat loss landscapes.
- **RMSProp:** Adapts the learning rate by dividing by a running average of recent gradient magnitudes. Excellent for non-stationary objectives.

- **AdaGrad:** Adapts the learning rate for each parameter individually based on historical gradients. Useful for sparse data but can stall prematurely.
- **Adam (Adaptive Moment Estimation):** Combines the momentum aspect of SGD with the adaptive scaling of RMSProp. It proved to be the most efficient optimizer for this project, converging rapidly with minimal tuning.